

Caso Netflix: questões para discussão

Aluno: Pedro Artur Jovanuzzi

Disciplina: Arquitetura de Software, Módulo 3: Serviços

Caso: Netflix: sete anos para reconstruir, dois segundos para começar o filme

Data: 01/09/2026

1. Segundo o anúncio oficial, o que aconteceu em 2008 e qual operação da empresa ficou parada por três dias?

Em agosto de 2008 o banco de dados relacional da Netflix foi corrompido. Era um banco único, escalado verticalmente, rodando no datacenter da própria empresa. O anúncio oficial de 2016 descreve o episódio em uma frase: "We experienced a major database corruption and for three days could not ship DVDs to our members."

A operação que parou foi o envio de DVDs pelo correio, que naquele momento ainda era o principal negócio da empresa. Foram três dias sem despachar disco nenhum.

O que mais chama atenção não é a duração, e sim o fato de tudo ter parado junto. Não houve degradação parcial, nem uma região afetada, nem um grupo de clientes atingido enquanto os outros seguiam sendo atendidos. Existia um componente só, e a perda dele levou a operação inteira embora.

A empresa tirou duas conclusões do episódio. A primeira é que insistir em escalar verticalmente um banco único não ia dar certo. A segunda, que costuma ser esquecida quando se conta essa história, é que comprar e instalar servidores no ritmo em que o streaming crescia era um problema que a Netflix não queria ter. Uma conclusão levou à reescrita da arquitetura, a outra levou à decisão de sair do datacenter próprio. São coisas diferentes, e a migração só faz sentido quando se lê as duas juntas.

2. A Netflix declara ter reconstruído a tecnologia em vez de transportá-la. Explique a diferença entre as duas coisas e o que cada uma muda na arquitetura resultante.

Transportar é pegar os sistemas do jeito que estão e colocá-los na infraestrutura de um provedor. Muda o dono do datacenter e muda a forma de pagar, porque em vez de comprar servidores a empresa passa a alugar capacidade sob demanda. A arquitetura em si continua a mesma: o banco central continua central, o monólito continua monólito e o ponto único de falha de 2008 continua existindo, só que agora hospedado na AWS. Ganha-se agilidade para provisionar máquina, mas não se ganha disponibilidade.

Reconstruir foi o que a Netflix diz ter feito ao adotar "a cloud-native approach, rebuilding virtually all of our technology". O banco relacional central deu lugar a armazenamentos NoSQL, cada um pertencendo a um serviço, e o monólito foi quebrado em serviços independentes. O projeto passou a assumir como premissa que instâncias, zonas e até regiões

inteiras falham.

Na arquitetura que sai de cada caminho, a diferença aparece em pontos bem concretos. Com banco compartilhado, a separação dos dados depende da disciplina de quem escreve o código; com um banco por serviço, ela vira característica da própria estrutura. Com um componente central, qualquer falha nele derruba o conjunto; com serviços separados, o sistema degrada em partes e continua atendendo o que não depende do pedaço quebrado. E a unidade de mudança deixa de ser o sistema inteiro e passa a ser um serviço de cada vez.

O preço disso foi tempo. A migração começou em 2008 e só terminou em janeiro de 2016, sete anos depois. Em troca, os resultados que a empresa divulga vieram da reescrita, e não da troca de provedor: oito vezes mais assinantes de streaming do que em 2008, visualização crescendo três ordens de grandeza em oito anos, disponibilidade perto de quatro noventa e custo de nuvem por início de reprodução equivalente a uma fração do que se gastava em datacenter próprio. Foi também a operação em várias regiões, que é consequência da reconstrução, que permitiu ligar o serviço em mais de 130 países de uma vez só, em 6 de janeiro de 2016.

A lição que dá para levar para outros projetos é que a nuvem entrega elasticidade e um modelo de custo diferente. Disponibilidade, escalabilidade horizontal e resiliência continuam vindo da arquitetura. Quem apenas transporta leva os próprios defeitos junto e passa a pagar aluguel por eles.

3. O Open Connect inverte a lógica de cache: o conteúdo chega antes do pedido. Que propriedade do negócio da Netflix torna isso possível, e que tipo de serviço jamais conseguiria fazer o mesmo?

Um cache comum é reativo. O primeiro usuário que pede o arquivo paga o custo de buscá-lo na origem e quem vem depois aproveita. O Open Connect trabalha ao contrário: a documentação oficial descreve o preenchimento noturno dos aparelhos e a pré-carga do conteúdo apropriado para a região de destino, num processo que leva de uma a duas semanas na configuração inicial de cada equipamento. Quando o assinante aperta o play, o vídeo não está sendo buscado, ele já estava a poucos quilômetros dali.

A propriedade do negócio que sustenta isso é o catálogo ser finito, conhecido com antecedência e estável no curto prazo. O conteúdo é pré-gravado e licenciado antes de entrar no ar, então existe uma lista fechada de arquivos, e ela não muda entre o preenchimento da madrugada e o consumo da noite seguinte. Some-se a isso uma demanda previsível: uma parte pequena do catálogo responde por boa parte das horas assistidas, e o padrão se repete de forma parecida dentro de cada região. É isso que mantém a taxa de acerto alta mesmo num aparelho de capacidade limitada, de até 120 TB no modelo de armazenamento e 60 TB no modelo global. Vale lembrar ainda que a recomendação influencia cerca de 80% das horas assistidas, ou seja, a empresa não só prevê o que vai ser assistido como participa dessa decisão.

Serviços em que o conteúdo não existe antes do pedido nunca conseguiriam fazer o mesmo. Transmissão ao vivo, como esporte e notícia, é o exemplo mais direto: o próximo segundo de

vídeo está sendo produzido agora, não há o que pré-carregar de madrugada. O mesmo vale para videochamada e mensageria, em que o conteúdo é criado pelos participantes no instante do uso. Conteúdo gerado por usuário também não fecha a conta na cauda longa, porque um vídeo publicado há dez minutos, ou um vídeo com cinco visualizações por mês, não justifica ocupar espaço em milhares de aparelhos. E respostas personalizadas ou transacionais, como busca, feed, saldo bancário e carrinho de compras, dependem de quem perguntou e de quando, então são calculadas por requisição.

O critério, no fim, é sempre o mesmo: cache proativo só compensa quando o conjunto de objetos é pequeno, conhecido antes do pedido e a demanda é concentrada o bastante para garantir taxa de acerto alta. É uma decisão que se apoia numa característica do negócio da Netflix, e não numa tecnologia. Por isso ela não se copia junto com o software.

4. Compare Isthmus, arquitetura ativa-ativa e Chaos Kong quanto ao modo de falha que cada um endereça.

As três coisas costumam aparecer juntas quando se fala da resiliência da Netflix, mas resolvem problemas diferentes e vieram em sequência, uma dependendo da outra.

O Isthmus, de 2013, ataca um modo de falha específico e já conhecido: a queda do balanceador de carga de uma região. A solução é rotear o tráfego de entrada por outra região enquanto o processamento continua na região original. É uma resposta limitada, feita sob medida para um problema, e não resolve a perda da região. Se a região inteira sumisse, o Isthmus não teria para onde mandar o trabalho.

A arquitetura ativa-ativa, de dezembro de 2013, tem outra natureza. Ela não é um remendo para um modo de falha, é uma capacidade que a plataforma passa a ter. O serviço roda em mais de uma região da AWS ao mesmo tempo e cada região consegue atender os membros da outra. Para isso funcionar foi preciso resolver problemas que não existem quando se tem uma região só: replicação do Cassandra entre regiões, invalidação de cache remoto no EVCache quando a escrita acontece do outro lado e roteamento no Zuul capaz de mudar em tempo de execução. O que se ganha é cobertura para qualquer falha regional, inclusive as que ninguém tinha previsto, e não apenas para a queda do balanceador.

O Chaos Kong não endereça modo de falha nenhum. Ele é o ensaio. Evacua uma região inteira da AWS com tráfego real rodando e observa se a outra absorve, o que serve para verificar na prática se a resiliência que se acredita ter existe mesmo. Só passou a ser possível depois da ativa-ativa, porque não dá para ensaiar a perda de uma região quando só existe uma. E foi justamente o ensaio que revelou o problema seguinte, que no papel não aparecia: evacuar uma região levava cerca de 50 minutos, tempo demais para um incidente real. O Project Nimble baixou esse número para 8 minutos, com uma equipe de duas pessoas em aproximadamente seis meses.

Comparando os três, então: o Isthmus cobre a falha de um componente conhecido, a ativa-ativa cobre a perda de qualquer componente regional, inclusive a região toda, e o Chaos Kong mede se as duas coisas anteriores realmente funcionam. Cada etapa depende da anterior, e é por isso que resumir tudo a "eles têm o Chaos Monkey" acaba escondendo uma evolução que levou cinco anos.

5. O Chaos Monkey exige a plataforma Spinnaker e o Hystrix está em modo de manutenção desde 2018. Explique o que cada um desses fatos diz sobre reaproveitar a plataforma de ferramentas de outra empresa.

O repositório do Chaos Monkey traz uma restrição que quase nunca aparece nos resumos: "You must be managing your apps with Spinnaker to use Chaos Monkey to terminate instances." A ferramenta assume a plataforma de entrega contínua da Netflix e, com ela, uma série de premissas que ninguém escreve no material de divulgação: instâncias imutáveis e descartáveis, grupos de auto scaling capazes de repor o que foi encerrado, serviços sem estado local, observabilidade suficiente para perceber o efeito de uma máquina morta e uma equipe capaz de responder ao que a ferramenta encontrar. Encerrar instâncias em produção só é uma prática defensável quando essas condições valem. Sem elas, o mesmo comando é só uma queda provocada pela própria empresa. Quem instala o Chaos Monkey sem o resto da fundação está copiando o efeito visível de uma engenharia que não tem.

O caso do Hystrix mostra outro lado do mesmo problema. A biblioteca virou padrão de fato para disjuntor de circuito no ecossistema Java e está em modo de manutenção desde novembro de 2018, com o próprio repositório indicando alternativas ativas como o Resilience4j. A empresa que abriu o código seguiu adiante quando as necessidades dela mudaram, e não tem obrigação nenhuma com quem adotou a biblioteca no meio do caminho. Adotar componente de terceiro é assumir um risco de manutenção que não entra na conta no dia da decisão.

Juntando os dois fatos, o que fica é que a plataforma de ferramentas de uma empresa é o subproduto do contexto dela, e não um produto pronto para levar. O que dá para reaproveitar de verdade é o raciocínio: por que a Netflix precisou de um disjuntor, que problema o Chaos Monkey resolve, que condições tornam essa prática segura. O padrão sobrevive, seja ele disjuntor de circuito, isolamento de recursos, prazo de espera ou degradação com resposta alternativa. A implementação é substituível, e o Resilience4j entrando no lugar do Hystrix mostra isso bem.

Também vale desconfiar do prazo de validade dos casos de referência. A plataforma de 2012 ainda é apresentada em palestras como se fosse o estado atual da engenharia da Netflix, anos depois de partes dela terem sido aposentadas. Copiar esse retrato é correr o risco de trazer respostas para perguntas que a sua empresa talvez nem tenha, e numa escala em que o remédio sai mais caro que a doença.

Fontes usadas

- Netflix, Completing the Netflix Cloud Migration, fevereiro de 2016.
- Netflix, páginas oficiais do programa Open Connect e das especificações dos aparelhos.
- Netflix Technology Blog: Isthmus, Active-Active for Multi-Regional Resiliency e Project Nimble.
- Repositórios oficiais do Chaos Monkey e do Hystrix, de onde vêm as duas citações e o aviso de modo de manutenção.
- Carlos A. Gomez-Uribe e Neil Hunt, The Netflix Recommender System: Algorithms, Business Value, and Innovation, ACM TMIS, dezembro de 2015.
- Página do caso na disciplina:
marco-mendes.github.io/arquitetura-software/modulo-3-servicos/casos-reais/